Assignment 1 _ UML Diagram

Name: Gia Bach Pham

Email: a3160100@adelaide.edu.au

Student ID: 3160100

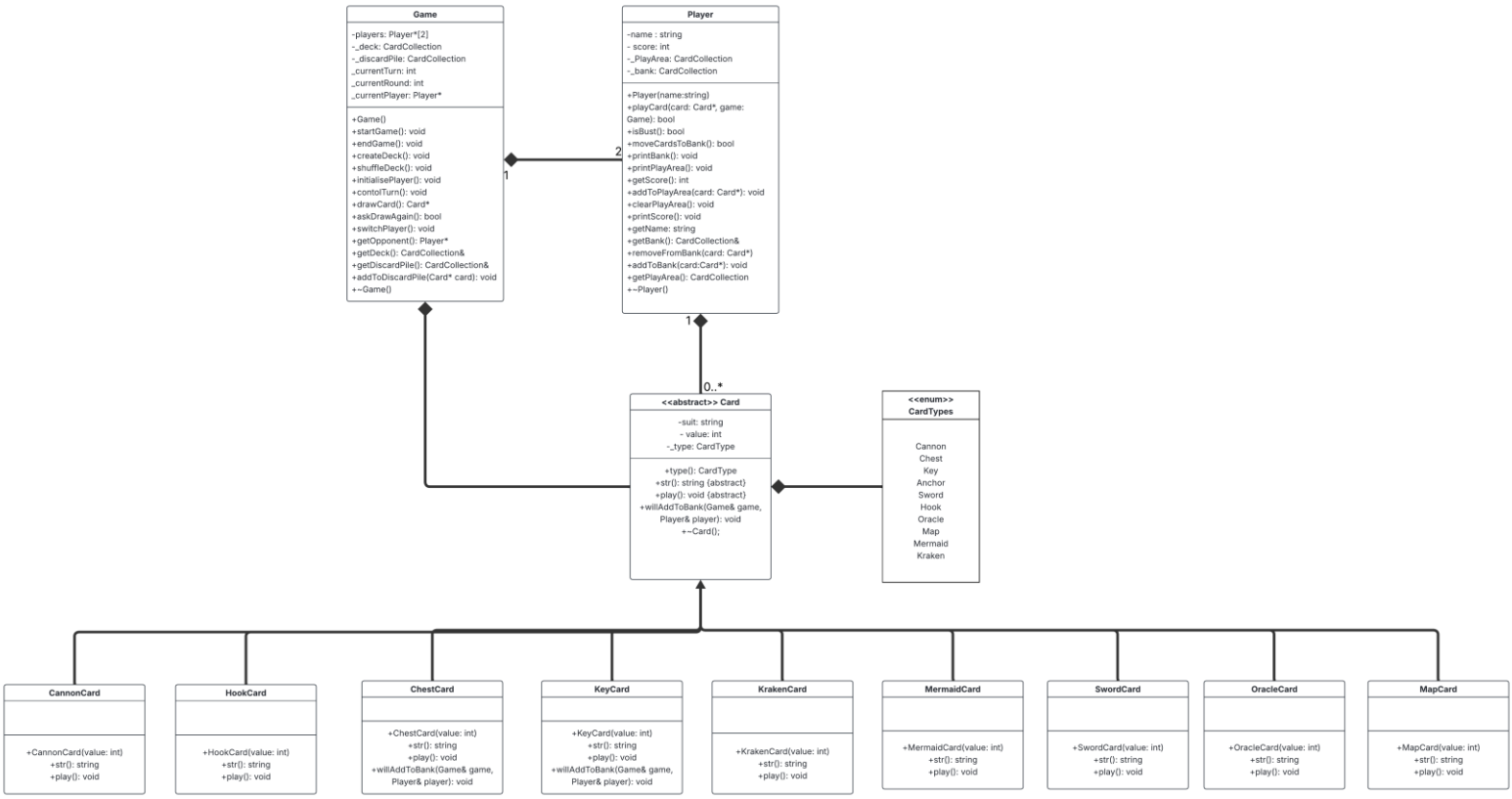
Key Design Decisions:

1. In Card Class: the willAddToBank() method is not an abstract class since its only applied for the key and chest classes
2. In Key and Chest Classes: the willAddToBank() method will be override
3. Card is an abstract base class: since each card type has its own unique abilites, pure virtual would force each class to implement its own behaviour
4. Play() takes Game& and Player& : allow the player to modify states in the game without coupling Card to Game, keep them looksely coupled
5. WillAddToBank(): not pure virtual, base cards provide empty defaults so other class doesnt have to override
6. Private fields and public methods enforces encapsulation
7. Bank and PlayArea are private, so its modified through methods like addToPlayArea, removeFromBank,...
8. Game orchestrate all interaction, player and cards dont directly interact
9. **moveCardToBank() checks for Chest and Key** — Checks if both exist before triggering willAddToBank() once, avoiding double triggering if both cards are in play area.
10. **getOpponent() in Game** — Allows card abilities to access the other player without exposing the players array directly, maintaining encapsulation.
11. **Two separate print methods in Player** — printBank() and printPlayArea() are separate because they are called at different points during a turn.

CardTypes Organisation — Single File for All Card Types

All 9 card subclasses are declared in a single header file (CardTypes.h) and implemented in a single source file (CardTypes.cpp). This decision was made to keep the project structure simple and manageable. Since all card classes share the same pattern (constructor, str(), play()), grouping them together avoids creating 18 separate files while still maintaining clear separation from the base Card class. Comments are used throughout to clearly separate each card type.

UML DIAGRAM


Game
-players: Player*[2]
-_deck: CardCollection
-_discardPile: CardCollection
_currentTurn: int
_currentRound: int
_currentPlayer: Player*
+Game()
+startGame(): void
+endGame(): void
+createDeck(): void
+shuffleDeck(): void
+initialisePlayer(): void
+contolTurn(): void
+drawCard(): Card*
+askDrawAgain(): bool
+switchPlayer(): void
+getOpponent(): Player*
+getDeck(): CardCollection&
+getDiscardPile(): CardCollection&
+addToDiscardPile(Card* card): void
+~Game()
1
2
Player
-name : string
- score: int
-_PlayArea: CardCollection
-_bank: CardCollection
+Player(name:string)
+playCard(card: Card*, game: Game): bool
+isBust(): bool
+moveCardsToBank(): bool
+printBank(): void
+printPlayArea(): void
+getScore(): int
+addToPlayArea(card: Card*): void
+clearPlayArea(): void
+printScore(): void
+getName: string
+getBank(): CardCollection&
+removeFromBank(card: Card*)
+addToBank(card:Card*): void
+getPlayArea(): CardCollection
+~Player()
1
0..*
<<abstract>> Card
-suit: string
- value: int
-_type: CardType
+type(): CardType
+str(): string {abstract}
+play(): void {abstract}
+willAddToBank(Game& game, Player& player): void
+~Card();
<<enum>>
CardTypes
Cannon
Chest
Key
Anchor
Sword
Hook
Oracle
Map
Mermaid
Kraken
CannonCard
+CannonCard(value: int)
+str(): string
+play(): void
HookCard
+HookCard(value: int)
+str(): string
+play(): void
ChestCard
+ChestCard(value: int)
+str(): string
+play(): void
+willAddToBank(Game& game, Player& player): void
KeyCard
+KeyCard(value: int)
+str(): string
+play(): void
+willAddToBank(Game& game, Player& player): void
KrakenCard
+KrakenCard(value: int)
+str(): string
+play(): void
MermaidCard
+MermaidCard(value: int)
+str(): string
+play(): void
SwordCard
+SwordCard(value: int)
+str(): string
+play(): void
OracleCard
+OracleCard(value: int)
+str(): string
+play(): void
MapCard
+MapCard(value: int)
+str(): string
+play(): void